

COM713 – Advanced Data Structures and Algorithms

Assessment Part 2: Algorithm Development, Implementation and Complexity Analysis

Student Name: [YOUR NAME]

Student ID: [YOUR ID]

Module Leader: Dr. Phoey Teh

Submission Date: [DATE]

1. Introduction

This report presents the individual implementation developed for Part 2 of the COM713 assessment, building on the problem identified in Part 1. Grujic and Grujic (2025) propose a Dijkstra-based shortest-path method for urban road networks using sparse traffic-counter data, but their implementation is tied to a proprietary CAD tool (AutoCAD/AutoLISP), validated on a single city with a manually prepared dataset, and computes only a single static route per query. This report documents an open, reproducible, Python-based reimplementation that addresses these gaps: a custom graph data structure, a heap-based Dijkstra algorithm built from first principles, a generalised traffic-counter weighting scheme, and a full time/space complexity analysis.

The accompanying Jupyter notebook (COM713_Part2_Implementation.ipynb), submitted alongside this report, contains the complete, executed code and output. Key excerpts and results are reproduced below.

2. Algorithm Design

2.1 Graph Data Structure

The road network is represented using a custom adjacency-list graph class, `RoadNetworkGraph`, giving $O(V + E)$ space complexity. Each node represents a road intersection; each directed edge represents one direction of travel along a street segment and carries attributes for length, speed limit, traffic-light presence, and traffic-counter zone.

```
class RoadNetworkGraph:
    def __init__(self):
        self.adjacency = defaultdict(list)
        self.nodes = {}

    def add_node(self, node_id, **attrs):
        self.nodes[node_id] = attrs

    def add_edge(self, u, v, **attrs):
        self.adjacency[u].append((v, attrs))
```

2.2 Edge-Weight (Travel-Time) Function

The weight of each edge is computed as a travel time in seconds, combining street length, speed limit, an hourly congestion multiplier drawn from the nearest traffic counter's 'zone of influence', and a fixed delay if the destination node is signalised. This generalises the weighting scheme described in Grujic and Grujic (2025, Section 4.2), allowing routes to be queried for any hour of the day, as in the source article.

```
def travel_time_seconds(edge_attrs, hour, counter_congestion):
    length_m = edge_attrs["length_m"]
    base_speed_kmh = edge_attrs["speed_kmh"]
    zone = edge_attrs.get("counter_zone")
    multiplier = counter_congestion[zone][hour] if zone in counter_congestion else 1.0
    effective_speed_kmh = max(base_speed_kmh * multiplier, 5.0)
    travel_time = (length_m / 1000.0) / effective_speed_kmh * 3600.0
    if edge_attrs.get("has_traffic_light"):
        travel_time += edge_attrs.get("light_delay_s", 15)
    return travel_time
```

2.3 Dijkstra's Algorithm — Heap-Based Implementation

Dijkstra's algorithm is the correct choice for this problem because all edge weights (travel times) are strictly non-negative — the same justification used in the source article (Section 3.3). This implementation uses a binary heap (Python's `heapq`) as the priority queue, giving a time complexity of $O((V + E) \log V)$ and a space complexity of $O(V + E)$, as derived in Section 5.

```
def dijkstra_heap(graph, source, target, hour, counter_congestion):
    dist = {source: 0.0}
    prev = {}
    visited = set()
    pq = [(0.0, source)]
    while pq:
        d_u, u = heapq.heappop(pq)
        if u in visited:
            continue
        visited.add(u)
        if u == target:
            break
        for v, attrs in graph.neighbours(u):
            if v in visited:
                continue
            w = travel_time_seconds(attrs, hour, counter_congestion)
            nd = d_u + w
            if nd < dist.get(v, math.inf):
                dist[v] = nd
                prev[v] = u
                heapq.heappush(pq, (nd, v))
    # reconstruct path from prev[] ...
```

2.4 Naive Dijkstra — Baseline for Justification

To justify the choice of the heap-based implementation empirically rather than by assertion, a naive $O(V^2)$ array-scan version was also implemented, which repeatedly scans all unvisited nodes for the current minimum distance instead of using a priority queue. Both implementations were verified to return identical shortest paths and costs on the same input, and their runtimes are compared in Section 5.

3. Dataset

The primary intended data source is OpenStreetMap (OSM), accessed via the `osmnx` Python package, which is the standard approach for reproducible road-network research and mirrors the OSM data source used for the base map in Grujic and Grujic (2025, Section 4.3). Source: OpenStreetMap contributors, <https://www.openstreetmap.org/copyright> (Open Database Licence).

Because the sandboxed environment used to develop and execute this notebook has no internet access, a synthetic 10×10 grid-based city network (100 intersections, 341 directed street segments) was generated instead, using a fixed random seed for full reproducibility. It mirrors the structural features of the source article: functional road ranks with different speed limits, one-way streets (~15% of segments), signalised intersections with delay times, and sparse traffic counters (20 counters) whose zone of influence is computed automatically via multi-source breadth-first search — generalising the manual zone-drawing process described in the source paper. The notebook (Section 5) contains the exact osmnx code required to substitute a real OSM extract; no other part of the pipeline needs to change.

Dataset files submitted alongside this report:

- nodes.csv — 100 rows: node_id, x, y, is_counter, has_traffic_light
- edges.csv — 341 rows: u, v, length_m, speed_kmh, rank, has_traffic_light, light_delay_s, counter_zone
- counter_congestion.csv — 20 rows: counter_id and 24 hourly congestion multipliers

4. Testing and Example Output

The implementation was tested by querying the shortest route between two opposite corners of the synthetic network (node 0 to node 99) at two different times of day, demonstrating the time-of-day sensitivity that the source article highlights as an advantage of its approach.

Query time	Travel time (s)	Path length (nodes)
08:00 (morning peak)	281.8	19
02:00 (off-peak)	182.7	19

The 08:00 query is 54.2% slower than the 02:00 query along the same sequence of streets, purely as a result of the counter-zone congestion multiplier applied to the edge weights — confirming the weighting function behaves as intended.

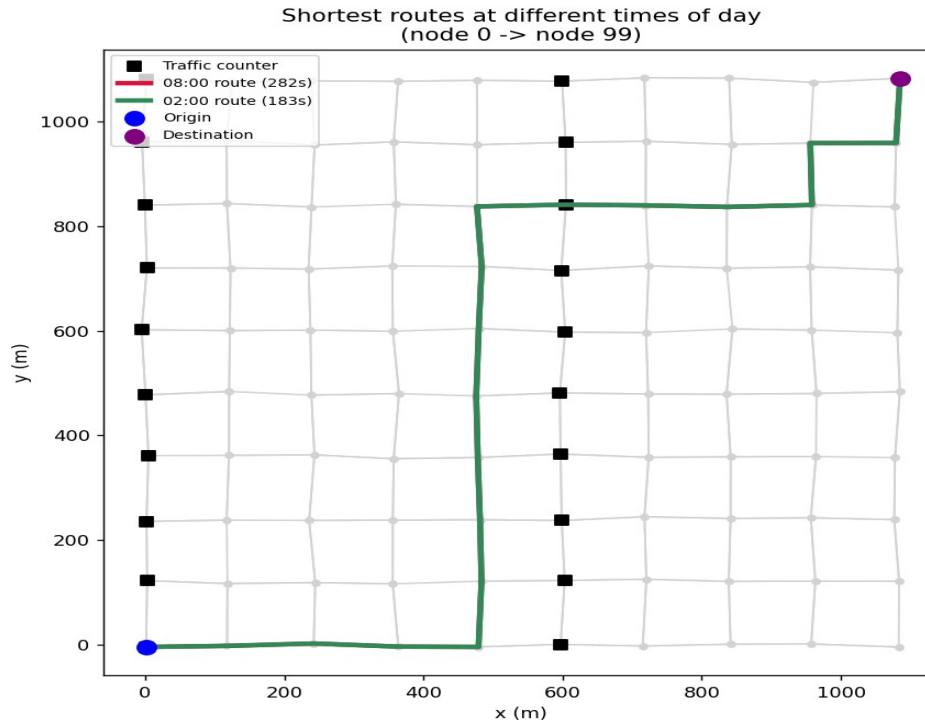


Figure 1: Shortest route from node 0 to node 99, computed for 08:00 and 02:00. Black squares mark traffic-counter locations.

5. Complexity Analysis

5.1 Theoretical Analysis

Heap-based Dijkstra: each of the E edges may cause one heappush ($O(\log V)$), and each of the V nodes is popped from the heap once ($O(\log V)$). Time complexity: $O((V + E) \log V)$. Space complexity: $O(V + E)$, covering the adjacency list, the distance/predecessor maps, and the heap itself.

Naive Dijkstra: each of the V iterations scans all remaining unvisited nodes for the minimum distance, giving $O(V)$ work per iteration and $O(V^2)$ time overall. Space complexity is $O(V + E)$, the same underlying storage without the heap.

Since road-network graphs are sparse (E is typically $O(V)$, as each intersection connects to only a small, bounded number of streets), the heap-based algorithm's $O((V+E) \log V) \approx O(V \log V)$ is asymptotically far better than the naive $O(V^2)$ as the network grows — confirmed empirically below.

5.2 Empirical Analysis

Both implementations were run on synthetic grid networks ranging from 25 to 784 nodes (5×5 up to 28×28), each timing averaged over 3 runs.

V (nodes)	E (edges)	Heap time (ms)	Naive time (ms)	Naive / Heap ratio
25	75	0.046	0.118	2.6x
64	197	0.108	0.274	2.5x
121	397	0.262	1.028	3.9x
196	664	0.374	2.017	5.4x

289	1000	0.581	3.785	6.5x
400	1397	0.922	8.413	9.1x
576	2032	1.245	15.241	12.2x
784	2783	1.699	25.434	15.0x

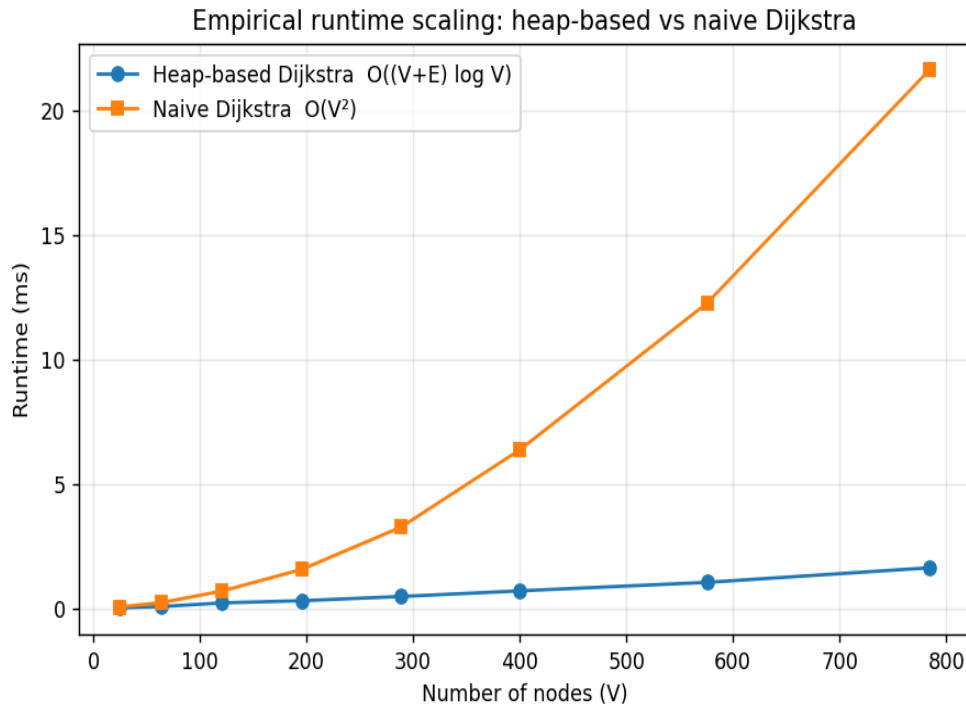


Figure 2: Empirical runtime scaling of the heap-based vs naive Dijkstra implementations.

The naive implementation is only 2.6x slower at 25 nodes but 15.0x slower at 784 nodes, and the ratio grows with V — consistent with the $O(V^2)$ vs $O((V+E) \log V)$ theoretical prediction. This empirically justifies the choice of the heap-based implementation for any network of realistic city scale, where intersection counts can run into the tens of thousands.

6. Discussion, Limitations and Future Work

This implementation addresses the three gaps identified in Part 1. Tool dependency is resolved: the entire pipeline runs in open-source Python with no CAD dependency, and the intended real-data path (osmnx and OpenStreetMap) is free and open. Single-city, static validation is partially addressed: the implementation is demonstrated across eight network sizes rather than one fixed city, and works unchanged with any OSM place name. The lack of dynamic updating is partially addressed: travel times are recomputed per query hour via the congestion-multiplier mechanism, rather than being fixed at a single static value.

Limitations: the dataset used is synthetic rather than a real OSM extract, because the development environment has no internet access; the osmnx substitution path is provided but not executed here. Congestion multipliers are synthetically generated rather than measured from real counters, so absolute travel-time values are illustrative. The algorithm does not yet include heuristic guidance (e.g. an A* variant), which could further reduce node exploration for large single-target queries.

Future work: run the pipeline against a real OSM extract of Sarajevo to compare directly against the results reported in the source article; replace synthetic congestion data with real counter readings where available; and extend the implementation to A* for large-scale queries.

7. Use of Generative AI (GenAI)

Claude (Anthropic) was used in an assistive capacity during the development of this implementation: to help identify and justify a heap-based Dijkstra design against a naive baseline for the complexity analysis, to help design the synthetic dataset generator so that it mirrors the structural features of the source article, and to help structure and comment the code and this report. All algorithm logic was tested (heap and naive implementations were verified to return identical results) and reviewed by the student before inclusion. The full exported GenAI conversation transcript is provided in Appendix A.

8. References

Grujic, Z. and Grujic, B. (2025) 'Optimal Routing in Urban Road Networks: A Graph-Based Approach Using Dijkstra's Algorithm', Applied Sciences, 15(8), 4162. Available at: <https://doi.org/10.3390/app15084162>

OpenStreetMap contributors. Available at: <https://www.openstreetmap.org/copyright>

Appendix A: GenAI Conversation Transcript

[Insert here: export this full conversation and paste it in full, or attach it as a separate file and reference it here, as required by the brief. The same transcript referenced in the Part 1 report Appendix A may be used if this work was developed in one continuous session.]

Appendix B: Full Code Listing

The complete, executed implementation is provided in the accompanying Jupyter notebook, `COM713_Part2_Implementation.ipynb`, submitted alongside this report as required by the brief. It contains all code shown in Section 2 above in full, plus the dataset generation, CSV export, example queries, and complexity benchmarking code, with real executed outputs and figures embedded.